

MongoDB & Mongoose SDE Interview Masterclass Notes

Kush Gupta – SDE Preparation

October 6, 2026

Contents

1	Phase 1: Foundations & Architecture (MongoDB Core)	2
1.1	NoSQL Philosophy and BSON	2
1.2	Anatomy of an ObjectId	2
1.3	CRUD Operations & Code Snippets	2
2	Phase 2: Schema Design & Advanced Modeling Patterns	3
2.1	Embedding vs. Referencing	3
2.2	Handling Relationships	3
2.3	Anti-patterns to Avoid	3
3	Phase 3: Performance, Indexing & Query Optimization	4
3.1	Index Types	4
3.2	The ESR Rule for Compound Indexes	4
3.3	Index Optimization Code Snippet	4
4	Phase 4: Aggregation Framework	5
4.1	Aggregation Pipeline Code Snippet	5
5	Phase 5: Mongoose ODM (Node.js Integration)	6
5.1	Schemas, Models, and Middleware	6
5.2	Mongoose Pre-Save Hook Snippet	6
6	Phase 6: Scaling, Transactions & Reliability	7
6.1	Replica Sets & High Availability	7
6.2	Multi-Document Transactions Code Snippet	7

1 Phase 1: Foundations & Architecture (MongoDB Core)

1.1 NoSQL Philosophy and BSON

MongoDB is a document-oriented NoSQL database designed for high scalability and flexibility. Unlike relational databases (SQL) that rely on rigid tabular structures, MongoDB stores dynamic schemas inside collections using documents.

- **Terminology Mapping:** Database → Database, Table → Collection, Row → Document, Column → Field.
- **BSON (Binary JSON):** MongoDB stores records as BSON. Unlike text JSON, BSON supports native data types like `ObjectId`, `Date`, and `Decimal128`, enabling rapid machine parsing.

1.2 Anatomy of an ObjectId

Every document requires a unique `_id` field, typically populated by a 12-byte BSON `ObjectId`:

- **Bytes 0–3:** Unix timestamp (seconds), ensuring chronological sortability.
- **Bytes 4–8:** 5-byte random value unique to the machine and process ID.
- **Bytes 9–11:** 3-byte incrementing counter.

1.3 CRUD Operations & Code Snippets

```
1 // 1. Efficient Read with Projection (Excluding _id)
2 db.products.find(
3   { category: "Electronics" },
4   { name: 1, price: 1, _id: 0 }
5 );
6
7 // 2. Atomic Upsert with Counter and Set-on-Insert
8 db.profileViews.updateOne(
9   { userId: "user_12345" },
10  {
11    $inc: { views: 1 },
12    $setOnInsert: { createdAt: new Date() }
13  },
14  { upsert: true }
15 );
```

Listing 1: Advanced CRUD operations with Projections and Upserts

2 Phase 2: Schema Design & Advanced Modeling Patterns

2.1 Embedding vs. Referencing

Schema design in MongoDB is workload-driven:

- **Embedding (Denormalization):** Stores related sub-documents inside a parent document. Maximizes read performance via atomic single-query retrieval but risks hitting the 16MB document size limit.
- **Referencing (Normalization):** Stores relationships via references (ObjectIDs) across collections, keeping document sizes small at the cost of requiring `$lookup` or application joins.

2.2 Handling Relationships

- **One-to-Few:** Embed directly (e.g., user addresses).
- **One-to-Many / Squillions:** Reference parent IDs in children (e.g., storing `authorId` in posts, or bucketing time-series telemetry data).

2.3 Anti-patterns to Avoid

Unbounded arrays (growing arrays endlessly inside a document) lead to heavy document reallocation overhead and eventual document size limit crashes.

3 Phase 3: Performance, Indexing & Query Optimization

3.1 Index Types

MongoDB provides Single Field, Compound, Multikey (for arrays), Text, and Geospatial (2dsphere) indexes to bypass full collection scans (COLLSCAN).

3.2 The ESR Rule for Compound Indexes

When building compound indexes, order fields using the **ESR Rule**:

1. **E (Equality)**: Fields tested with exact match conditions.
2. **S (Sort)**: Fields specified in sort clauses.
3. **R (Range)**: Fields evaluated with range operators (\$gt, \$lt).

3.3 Index Optimization Code Snippet

```
1 // Query: db.transactions.find({ userId: "user_999", status: "completed" }).sort
  ({ amount: -1 });
2 db.transactions.createIndex({ userId: 1, status: 1, amount: -1 });
```

Listing 2: Creating an ESR-compliant compound index

4 Phase 4: Aggregation Framework

The Aggregation Pipeline processes data sequentially across pipeline stages (\$match, \$group, \$project, \$sort, \$lookup).

4.1 Aggregation Pipeline Code Snippet

```
1 db.orders.aggregate([
2   { $match: { status: "completed" } },
3   {
4     $group: {
5       _id: "$customerId",
6       totalSpent: { $sum: "$amount" }
7     }
8   },
9   { $sort: { totalSpent: -1 } },
10  { $limit: 3 }
11 ]);
```

Listing 3: Top 3 customers by spending using aggregation

5 Phase 5: Mongoose ODM (Node.js Integration)

5.1 Schemas, Models, and Middleware

Mongoose maps application objects to MongoDB documents, offering data validation, virtual properties, and middleware hooks.

5.2 Mongoose Pre-Save Hook Snippet

```
1 const bcrypt = require('bcrypt');
2
3 userSchema.pre("save", async function(next) {
4   if (!this.isModified("password")) return next();
5   try {
6     const salt = await bcrypt.genSalt(10);
7     this.password = await bcrypt.hash(this.password, salt);
8     next();
9   } catch (err) {
10    next(err);
11  }
12 });
```

Listing 4: Conditional password hashing via pre-save hook

6 Phase 6: Scaling, Transactions & Reliability

6.1 Replica Sets & High Availability

Replica sets consist of a Primary node (handles writes) and Secondary nodes (replicate via the Oplog). Automatic failover kicks in via elections if the primary drops.

6.2 Multi-Document Transactions Code Snippet

```
1 const session = await mongoose.startSession();
2 session.startTransaction();
3
4 try {
5   const opts = { session };
6   await Account.updateOne({ accountNumber: "ACC_A" }, { $inc: { balance: -100 } }, opts);
7   await Account.updateOne({ accountNumber: "ACC_B" }, { $inc: { balance: 100 } }, opts);
8
9   await session.commitTransaction();
10 } catch (error) {
11   await session.abortTransaction();
12 } finally {
13   session.endSession();
14 }
```

Listing 5: ACID transactions across multiple documents using Mongoose sessions